

R-Code Beispiele aus WDDA Vorlesung 7 (Kapitel 7)

WDDA – Datenmanagement und Datenanalyse

Prof. Dr. Ulrich Matter

2026-05-20

Inhaltsverzeichnis

1	Einleitung	1
2	Vorbereitung	2
3	Bootstrap und Konfidenzintervalle	2
3.1	Einfache lineare Regression	2
3.2	Visualisierung der Bootstrap-Verteilungen	4
3.3	Multiple Regression	5
3.4	Visualisierung der Bootstrap-Verteilungen für multiple Regression	6
4	Inferenz und Signifikanztests	7
4.1	t-Tests für Regressionskoeffizienten	7
4.2	F-Statistik und Modellvergleich	8
4.3	Prognose mit Konfidenz- und Prognoseintervallen	9
4.4	Visualisierung von Konfidenz- und Prognoseintervallen	10
5	Zusammenfassung	12

1 Einleitung

Dieses Dokument fasst alle R-Code-Beispiele der Folien aus der WDDA FS2026 Vorlesung 7 (Kapitel 7) zusammen. Der Fokus liegt auf der Inferenz in Regressionsmodellen, insbesondere auf Konfidenzintervallen, Signifikanztests und Prognosen.

Die statistische Inferenz ermöglicht es uns, von einer Stichprobe auf die Grundgesamtheit zu schliessen. In der Regressionsanalyse interessieren wir uns für die Unsicherheit der geschätzten Koeffizienten und die Zuverlässigkeit unserer Vorhersagen.

Zentrale Konzepte in diesem Bereich sind:

1. **Konfidenzintervalle für Regressionskoeffizienten:** Wie genau sind unsere Schätzungen?
2. **Bootstrap-Methoden:** Wie können wir die Verteilung der Koeffizienten simulieren?
3. **Signifikanztests:** Sind die beobachteten Effekte statistisch signifikant?
4. **Prognoseintervalle:** Wie zuverlässig sind unsere Vorhersagen?

2 Vorbereitung

```
# Pakete laden
library(mosaic)      # für do(), resample()
library(ggplot2)     # für Visualisierungen
library(readxl)      # zum Einlesen von Excel-Dateien

# Daten laden
Advertising <- read_excel("data/WDDA_07.xlsx", sheet = "Advertising")

## tibble [200 x 4] (S3: tbl_df/tbl/data.frame)
## $ TV          : num [1:200] 230.1 44.5 17.2 151.5 180.8 ...
## $ radio       : num [1:200] 37.8 39.3 45.9 41.3 10.8 48.9 32.8 19.6 2.1 2.6 ...
## $ newspaper: num [1:200] 69.2 45.1 69.3 58.5 58.4 75 23.5 11.6 1 21.2 ...
## $ sales      : num [1:200] 22.1 10.4 9.3 18.5 12.9 7.2 11.8 13.2 4.8 10.6 ...
```

R-Erklärung:

- Die Funktion `library()` lädt ein installiertes Paket in die aktuelle R-Sitzung.
- Das Paket `mosaic` bietet Funktionen für statistische Simulationen, insbesondere `do()` und `resample()`.
- Das Paket `ggplot2` ermöglicht die Erstellung von ansprechenden Visualisierungen.
- Mit `read_excel()` aus dem Paket `readxl` können Excel-Dateien eingelesen werden.
- Die Funktion `str()` zeigt die Struktur eines Objekts, einschliesslich Datentypen und Dimensionen.

Hinweis zu Datenpfaden: In R-Markdown-Dateien werden relative Pfade vom Speicherort der Datei aus interpretiert, daher der Pfad `../data/WDDA_07.xlsx`. In regulären R-Skripten würde der Pfad relativ zum Arbeitsverzeichnis angegeben werden.

3 Bootstrap und Konfidenzintervalle

Die Bootstrap-Methode ist ein Resampling-Verfahren, das es ermöglicht, die Stichprobenverteilung einer Statistik zu schätzen, ohne Annahmen über die zugrundeliegende Verteilung zu treffen.

3.1 Einfache lineare Regression

```
# Einfaches Regressionsmodell (TV)
mdl <- lm(sales ~ TV, data = Advertising)
summary(mdl)

##
## Call:
## lm(formula = sales ~ TV, data = Advertising)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.3860 -1.9545 -0.1913  2.0671  7.2124
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.032594   0.457843   15.36  <2e-16 ***
## TV           0.047537   0.002691   17.67  <2e-16 ***
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.259 on 198 degrees of freedom
## Multiple R-squared:  0.6119, Adjusted R-squared:  0.6099
## F-statistic: 312.1 on 1 and 198 DF,  p-value: < 2.2e-16
```

```
# Klassisches Konfidenzintervall
confint(mdl)
```

```
##              2.5 %      97.5 %
## (Intercept) 6.12971927 7.93546783
## TV          0.04223072 0.05284256
```

```
# Bootstrap-Simulation für Koeffizienten
set.seed(123) # für Reproduzierbarkeit
simcoef <- do(5000) * coef(lm(sales ~ TV, data = resample(Advertising)))
CI_intercept <- quantile(simcoef$Intercept, c(0.025, 0.975))
CI_tv <- quantile(simcoef$TV, c(0.025, 0.975))
cat("Bootstrap 95%-CI für Intercept:", round(CI_intercept, 3), "\n")
```

```
## Bootstrap 95%-CI für Intercept: 6.391 7.697
```

```
cat("Bootstrap 95%-CI für Steigung (TV):", round(CI_tv, 3), "\n\n")
```

```
## Bootstrap 95%-CI für Steigung (TV): 0.042 0.053
```

Statistische Erklärung:

- **Konfidenzintervalle** geben einen Bereich an, in dem der wahre Parameterwert mit einer bestimmten Wahrscheinlichkeit (z.B. 95%) liegt.
- Die **Bootstrap-Methode** erstellt viele Stichproben durch Ziehen mit Zurücklegen aus der ursprünglichen Stichprobe.
- Für jede Bootstrap-Stichprobe wird das Regressionsmodell neu angepasst und die Koeffizienten werden gespeichert.
- Die Verteilung dieser Koeffizienten approximiert die Stichprobenverteilung der Schätzer.
- Das **95%-Konfidenzintervall** wird durch die 2,5%- und 97,5%-Quantile dieser Verteilung bestimmt.
- Im Vergleich zum klassischen Konfidenzintervall (basierend auf der t-Verteilung) macht die Bootstrap-Methode weniger Annahmen über die Verteilung der Daten.

R-Erklärung:

- Die Funktion `lm()` (linear model) passt ein lineares Regressionsmodell an die Daten an.
- Die Funktion `confint()` berechnet Konfidenzintervalle für die Koeffizienten eines Regressionsmodells basierend auf der t-Verteilung.
- Die Funktion `set.seed()` setzt den Startwert für den Zufallszahlengenerator, um reproduzierbare Ergebnisse zu erhalten.
- Die Funktion `resample()` aus dem Paket `mosaic` zieht eine Stichprobe mit Zurücklegen aus den Daten.
- Die Funktion `do()` aus dem Paket `mosaic` wiederholt einen Ausdruck mehrmals und sammelt die Ergebnisse.
- Die Funktion `quantile()` berechnet Quantile einer Verteilung.

3.2 Visualisierung der Bootstrap-Verteilungen

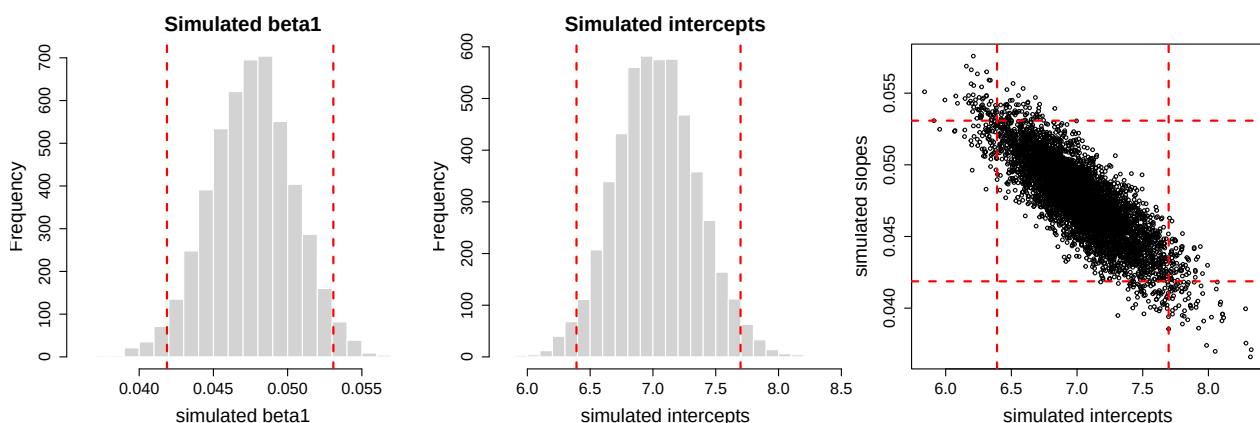
```
# Visualisierung der Bootstrap-Verteilungen
par(mfrow = c(1, 3), cex.main = 1.8, cex.lab = 1.6, cex.axis = 1.4, mar = c(4, 4, 3, 1))

# Gemeinsame Achsenlimits definieren
tv_range <- range(simcoef$TV)
intercept_range <- range(simcoef$Intercept)

# Histogramm für Steigung (beta1)
hist(simcoef$TV, main = "Simulated beta1", xlab = "simulated beta1",
     col = "lightgray", border = "white", breaks = 20,
     xaxt = "n") # Keine Achsenbeschriftung zunächst
axis(1, at = axTicks(1)) # Explizite Achsenbeschriftung
tv_ticks <- axTicks(1)
abline(v = quantile(simcoef$TV, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

# Histogramm für Intercept (beta0)
hist(simcoef$Intercept, main = "Simulated intercepts", xlab = "simulated intercepts",
     col = "lightgray", border = "white", breaks = 20,
     xaxt = "n") # Keine Achsenbeschriftung zunächst
axis(1, at = axTicks(1)) # Explizite Achsenbeschriftung
intercept_ticks <- axTicks(1)
abline(v = quantile(simcoef$Intercept, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

# Scatterplot der Koeffizienten
plot(simcoef$Intercept, simcoef$TV, pch = 1, cex = 0.7,
     xlab = "simulated intercepts", ylab = "simulated slopes",
     xaxt = "n", yaxt = "n") # Keine Achsenbeschriftung zunächst
axis(1, at = intercept_ticks) # Gleiche Ticks wie im Intercept-Histogramm
axis(2, at = tv_ticks) # Gleiche Ticks wie im TV-Histogramm
abline(v = quantile(simcoef$Intercept, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)
abline(h = quantile(simcoef$TV, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)
```



```
par(mfrow = c(1, 1))
```

Statistische Erklärung:

- Die **Histogramme** zeigen die Verteilung der Bootstrap-Schätzungen für den Achsenabschnitt (Intercept) und die Steigung (TV-Koeffizient).
- Die **roten gestrichelten Linien** markieren die Grenzen des 95%-Konfidenzintervalls.
- Der **Scatterplot** zeigt die gemeinsame Verteilung der beiden Koeffizienten.
- Die **negative Korrelation** zwischen Achsenabschnitt und Steigung ist typisch für Regressionsmodelle und entsteht durch die Struktur der Daten und die Schätzmethode.
- Diese Visualisierungen helfen, die Unsicherheit der Schätzungen und die Beziehungen zwischen den Koeffizienten zu verstehen.

R-Erklärung:

- Die Funktion `par()` mit dem Parameter `mfrow = c(1, 3)` teilt das Grafikfenster in 1 Zeile und 3 Spalten.
- Die Parameter `cex.main`, `cex.lab` und `cex.axis` in `par()` steuern die Schriftgrösse der Titel, Achsenbeschriftungen und Achsenwerte.
- Die Funktion `hist()` erstellt ein Histogramm.
- Die Funktion `abline()` mit dem Parameter `v` fügt vertikale Linien zum Plot hinzu.
- Die Funktion `plot()` erstellt ein Streudiagramm.

3.3 Multiple Regression

```
# Multiple Regression (TV + radio)
md2 <- lm(sales ~ TV + radio, data = Advertising)
summary(md2)
```

```
##
## Call:
## lm(formula = sales ~ TV + radio, data = Advertising)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.7977 -0.8752  0.2422  1.1708  2.8328
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.92110    0.29449   9.919  <2e-16 ***
## TV             0.04575    0.00139  32.909  <2e-16 ***
## radio         0.18799    0.00804  23.382  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.681 on 197 degrees of freedom
## Multiple R-squared:  0.8972, Adjusted R-squared:  0.8962
## F-statistic: 859.6 on 2 and 197 DF,  p-value: < 2.2e-16
```

```
confint(md2)
```

```
##              2.5 %      97.5 %
## (Intercept) 2.34034299 3.50185683
## TV          0.04301292 0.04849671
## radio       0.17213877 0.20384969
```

```
# Bootstrap für multiple Regression
set.seed(456)
simcoef_multi <- do(5000) * coef(lm(sales ~ TV + radio, data = resample(Advertising)))
CI_tv_multi <- quantile(simcoef_multi$TV, c(0.025, 0.975))
CI_radio_multi <- quantile(simcoef_multi$radio, c(0.025, 0.975))
cat("95%-CI für TV (multi):", round(CI_tv_multi, 3), "\n")
```

```
## 95%-CI für TV (multi): 0.042 0.05
```

```
cat("95%-CI für Radio (multi):", round(CI_radio_multi, 3), "\n\n")
```

```
## 95%-CI für Radio (multi): 0.168 0.207
```

Statistische Erklärung:

- In der **multiplen Regression** werden Konfidenzintervalle für jeden Koeffizienten berechnet, unter Berücksichtigung der anderen Prädiktoren im Modell.
- Die Interpretation eines Konfidenzintervalls in der multiplen Regression ist: "Wenn alle anderen Prädiktoren konstant gehalten werden, liegt der wahre Effekt des Prädiktors mit 95% Wahrscheinlichkeit in diesem Intervall."
- Die Bootstrap-Methode kann auch auf multiple Regressionsmodelle angewendet werden, um die Verteilung der Koeffizienten zu schätzen.
- Im Vergleich zur einfachen Regression können sich die Konfidenzintervalle für einen Prädiktor ändern, wenn andere Prädiktoren ins Modell aufgenommen werden, insbesondere wenn die Prädiktoren korreliert sind.

R-Erklärung:

- Die Formel `sales ~ TV + radio` in `lm()` gibt an, dass `sales` die abhängige Variable und `TV` und `radio` die unabhängigen Variablen sind.
- Der Ausdruck `simcoef_multi$TV` extrahiert die Spalte `TV` aus dem Dataframe `simcoef_multi`.

3.4 Visualisierung der Bootstrap-Verteilungen für multiple Regression

```
# Visualisierung der Bootstrap-Verteilungen für multiple Regression
par(mfrow = c(1, 3), cex.main = 1.8, cex.lab = 1.6, cex.axis = 1.4, mar = c(4, 4, 3, 1))

# Gemeinsame Achsenlimits definieren
tv_multi_range <- range(simcoef_multi$TV)
radio_multi_range <- range(simcoef_multi$radio)

# Histogramm für TV-Koeffizient
hist(simcoef_multi$TV, main = "TV coefficient", xlab = "simulated TV coef",
     col = "lightgray", border = "white", breaks = 20,
     xaxt = "n") # Keine Achsenbeschriftung zunächst
axis(1, at = axTicks(1)) # Explizite Achsenbeschriftung
tv_multi_ticks <- axTicks(1)
abline(v = quantile(simcoef_multi$TV, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

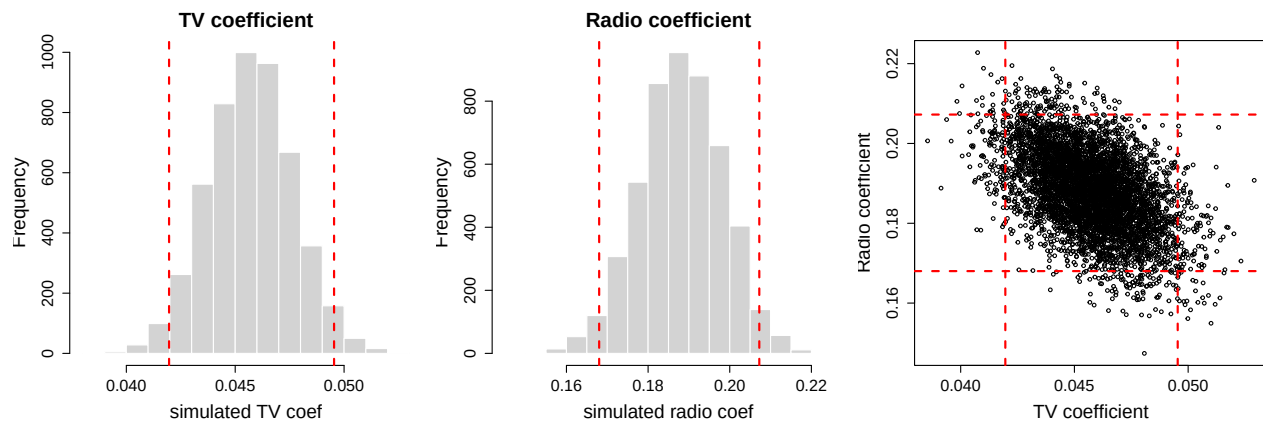
# Histogramm für Radio-Koeffizient
hist(simcoef_multi$radio, main = "Radio coefficient", xlab = "simulated radio coef",
     col = "lightgray", border = "white", breaks = 20,
     xaxt = "n") # Keine Achsenbeschriftung zunächst
```

```

axis(1, at = axTicks(1)) # Explizite Achsenbeschriftung
radio_multi_ticks <- axTicks(1)
abline(v = quantile(simcoef_multi$radio, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

# Scatterplot der Koeffizienten
plot(simcoef_multi$TV, simcoef_multi$radio, pch = 1, cex = 0.7,
     xlab = "TV coefficient", ylab = "Radio coefficient",
     xaxt = "n", yaxt = "n") # Keine Achsenbeschriftung zunächst
axis(1, at = tv_multi_ticks) # Gleiche Ticks wie im TV-Histogramm
axis(2, at = radio_multi_ticks) # Gleiche Ticks wie im Radio-Histogramm
abline(v = quantile(simcoef_multi$TV, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)
abline(h = quantile(simcoef_multi$radio, c(0.025, 0.975)), col = "red", lty = 2, lwd = 2)

```



```
par(mfrow = c(1, 1))
```

Statistische Erklärung:

- Die **Histogramme** zeigen die Verteilung der Bootstrap-Schätzungen für die TV- und Radio-Koeffizienten.
- Der **Scatterplot** zeigt die gemeinsame Verteilung der beiden Koeffizienten.
- Eine **Korrelation zwischen den Koeffizienten** kann auf Multikollinearität hindeuten, d.h. die Prädiktoren sind korreliert.
- Bei starker Multikollinearität können die Konfidenzintervalle breiter werden, was die Präzision der Schätzungen verringert.
- Die Visualisierung der gemeinsamen Verteilung kann helfen, komplexe Beziehungen zwischen den Koeffizienten zu erkennen, die in den einzelnen Konfidenzintervallen nicht sichtbar sind.

4 Inferenz und Signifikanztests

Signifikanztests ermöglichen es, die statistische Signifikanz der geschätzten Koeffizienten zu beurteilen und Hypothesen über die Parameter zu testen.

4.1 t-Tests für Regressionskoeffizienten

```

# t-Tests für Regressionskoeffizienten
cat("t-Werte und p-Werte für md2:\n")

```

t-Werte und p-Werte für md2:

```
print(summary(md2)$coefficients)
```

```
##              Estimate Std. Error  t value    Pr(>|t|)
## (Intercept) 2.92109991 0.294489678  9.919193 4.565557e-19
## TV          0.04575482 0.001390356 32.908708 5.436980e-82
## radio       0.18799423 0.008039973 23.382446 9.776972e-59
```

Statistische Erklärung:

- Der **t-Test** für einen Regressionskoeffizienten prüft die Nullhypothese $H_0: \beta = 0$ gegen die Alternativhypothese $H_1: \beta \neq 0$.
- Der **t-Wert** ist der Quotient aus dem geschätzten Koeffizienten und seinem Standardfehler: $t = \hat{\beta} / SE(\hat{\beta})$.
- Der **p-Wert** gibt die Wahrscheinlichkeit an, einen t-Wert zu beobachten, der mindestens so extrem ist wie der berechnete, wenn die Nullhypothese wahr ist.
- Ein **kleiner p-Wert** (typischerweise < 0.05) führt zur Ablehnung der Nullhypothese und deutet darauf hin, dass der Prädiktor einen signifikanten Einfluss auf die abhängige Variable hat.
- Die **Standardfehler** der Koeffizienten geben die Präzision der Schätzungen an. Kleinere Standardfehler bedeuten präzisere Schätzungen.
- In der multiplen Regression werden die t-Tests für jeden Koeffizienten durchgeführt, wobei die anderen Prädiktoren im Modell berücksichtigt werden.

R-Erklärung:

- Der Ausdruck `summary(md2)$coefficients` extrahiert die Koeffiziententabelle aus dem Regressionsobjekt.
- Diese Tabelle enthält die geschätzten Koeffizienten, ihre Standardfehler, t-Werte und p-Werte.

4.2 F-Statistik und Modellvergleich

```
# F-Statistik des Gesamtmodells
f_stat <- summary(md2)$fstatistic
f_value <- f_stat["value"]
df1 <- f_stat["numdf"]
df2 <- f_stat["dendf"]
cat("\nF-Statistik:", f_value, " (df1 =", df1, ", df2 =", df2, ")\n")
```

```
##
## F-Statistik: 859.6177 (df1 = 2 , df2 = 197 )
```

```
# Modellvergleich mit Zusatzprädiktor (Newspaper)
md3 <- lm(sales ~ TV + radio + newspaper, data = Advertising)
anova(md2, md3) # Vergleich mittels F-Test
```

```
## Analysis of Variance Table
##
## Model 1: sales ~ TV + radio
## Model 2: sales ~ TV + radio + newspaper
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1     197 556.91
## 2     196 556.83  1  0.088717 0.0312 0.8599
```


Statistische Erklärung:

- Die **F-Statistik** testet die Nullhypothese, dass alle Regressionskoeffizienten (ausser dem Achsenabschnitt) gleich Null sind, gegen die Alternativhypothese, dass mindestens ein Koeffizient ungleich Null ist.
- Die F-Statistik ist definiert als: $F = (ESS/p) / (RSS/(n-p-1))$, wobei ESS die erklärte Quadratsumme, RSS die Residuenquadratsumme, n die Anzahl der Beobachtungen und p die Anzahl der Prädiktoren ist.
- Ein **grosser F-Wert** und ein **kleiner p-Wert** deuten darauf hin, dass das Modell signifikant besser ist als ein Modell ohne Prädiktoren.
- Der **ANOVA F-Test** vergleicht zwei geschachtelte Modelle und prüft, ob das komplexere Modell signifikant besser ist als das einfachere.
- In diesem Beispiel vergleichen wir ein Modell mit TV und Radio als Prädiktoren mit einem Modell, das zusätzlich Newspaper enthält.
- Ein **nicht signifikanter F-Test** ($p > 0.05$) deutet darauf hin, dass der zusätzliche Prädiktor (Newspaper) keinen signifikanten Beitrag zur Erklärungskraft des Modells leistet.

R-Erklärung:

- Der Ausdruck `summary(md2)$fstatistic` extrahiert die F-Statistik aus dem Regressionsobjekt.
- Die Funktion `anova()` führt einen F-Test zum Vergleich zweier geschachtelter Modelle durch.
- Die Modelle müssen geschachtelt sein, d.h. alle Prädiktoren des einfacheren Modells müssen auch im komplexeren Modell enthalten sein.

4.3 Prognose mit Konfidenz- und Prognoseintervallen

```
# Prognose: Punktschätzung & Intervalle
neue_werte <- data.frame(TV = 230.1, radio = 37.8)
pred <- predict(md2, newdata = neue_werte, interval = "confidence") # Konfidenzintervall
pred2 <- predict(md2, newdata = neue_werte, interval = "prediction") # Prognoseintervall
cat("\nKonfidenzintervall für Erwartungswert (mean sales):\n")
```

```
##
## Konfidenzintervall für Erwartungswert (mean sales):
```

```
print(pred)
```

```
##          fit          lwr          upr
## 1 20.55546 20.16278 20.94815
```

```
cat("\nPrognoseintervall für Einzelbeobachtung:\n")
```

```
##
## Prognoseintervall für Einzelbeobachtung:
```

```
print(pred2)
```

```
##          fit          lwr          upr
## 1 20.55546 17.21652 23.89441
```

Statistische Erklärung:

- Die **Punktprognose** ist der vorhergesagte Wert der abhängigen Variable für bestimmte Werte der unabhängigen Variablen: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$

- Das **Konfidenzintervall für den Erwartungswert** gibt den Bereich an, in dem der wahre Erwartungswert mit einer bestimmten Wahrscheinlichkeit liegt.
- Das **Prognoseintervall** gibt den Bereich an, in dem eine einzelne neue Beobachtung mit einer bestimmten Wahrscheinlichkeit liegen wird.
- Prognoseintervalle sind **breiter** als Konfidenzintervalle für den Erwartungswert, da sie zusätzlich die Variation der einzelnen Beobachtungen um den Erwartungswert berücksichtigen.
- Die Breite beider Intervalle hängt von mehreren Faktoren ab:
 - Der Stichprobengröße: Größere Stichproben führen zu engeren Intervallen.
 - Der Variabilität der Daten: Höhere Variabilität führt zu breiteren Intervallen.
 - Der Position der Prädiktorwerte: Intervalle sind am engsten nahe dem Mittelwert der Prädiktoren und werden breiter, je weiter man sich davon entfernt.

R-Erklärung:

- Die Funktion `predict()` berechnet vorhergesagte Werte für ein Regressionsmodell.
- Der Parameter `newdata` in `predict()` gibt einen Dataframe mit den Werten der Prädiktoren an, für die Vorhersagen gemacht werden sollen.
- Der Parameter `interval` in `predict()` gibt an, ob Konfidenz- oder Prognoseintervalle berechnet werden sollen.
- Die Ausgabe enthält drei Spalten: `fit` (die Punktprognose), `lwr` (die untere Grenze des Intervalls) und `upr` (die obere Grenze des Intervalls).

4.4 Visualisierung von Konfidenz- und Prognoseintervallen

```
# Erstellen eines Gitters von TV-Werten für die Vorhersage
tv_grid <- seq(min(Advertising$TV), max(Advertising$TV), length.out = 100)
radio_mean <- mean(Advertising$radio)
newdata <- data.frame(TV = tv_grid, radio = radio_mean)

# Konfidenzintervall für den Erwartungswert
conf_int <- predict(md2, newdata = newdata, interval = "confidence")
conf_df <- data.frame(TV = tv_grid, fit = conf_int[, "fit"],
                     lwr = conf_int[, "lwr"], upr = conf_int[, "upr"])

# Prognoseintervall
pred_int <- predict(md2, newdata = newdata, interval = "prediction")
pred_df <- data.frame(TV = tv_grid, fit = pred_int[, "fit"],
                     lwr = pred_int[, "lwr"], upr = pred_int[, "upr"])

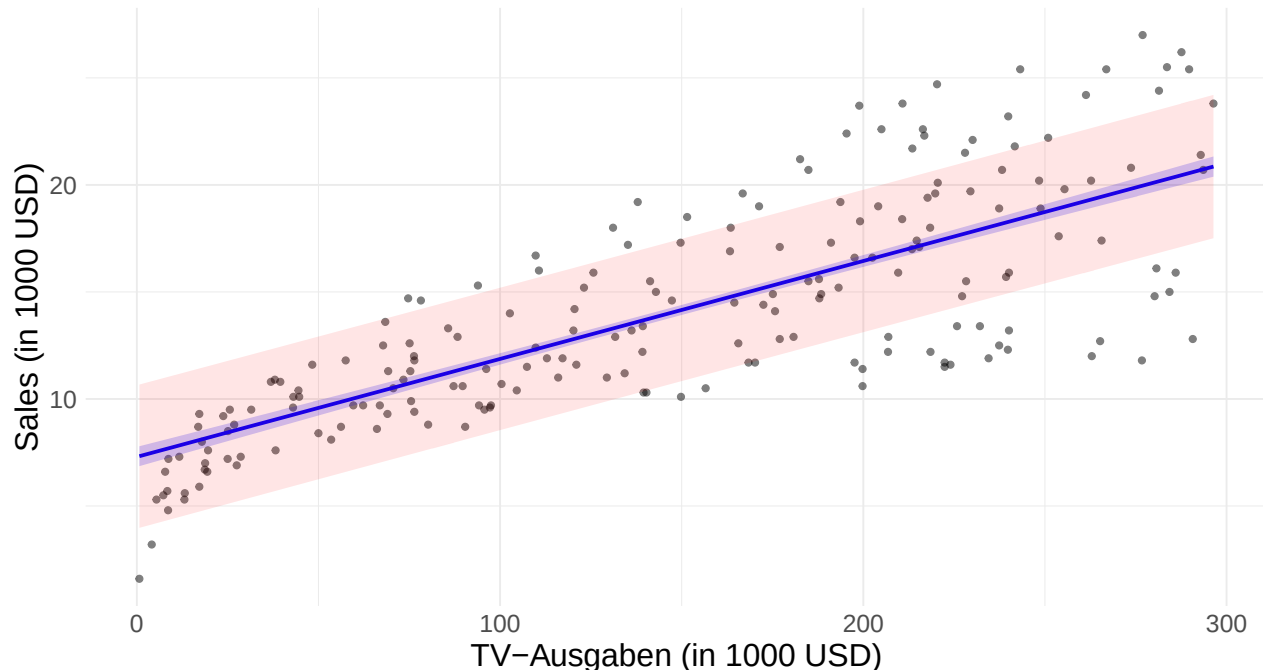
# Visualisierung
ggplot() +
  geom_point(data = Advertising, aes(x = TV, y = sales), alpha = 0.5) +
  geom_line(data = conf_df, aes(x = TV, y = fit), color = "blue", size = 1) +
  geom_ribbon(data = conf_df, aes(x = TV, ymin = lwr, ymax = upr),
            fill = "blue", alpha = 0.2) +
  geom_ribbon(data = pred_df, aes(x = TV, ymin = lwr, ymax = upr),
            fill = "red", alpha = 0.1) +
  labs(title = "Regressionslinie mit Konfidenz- und Prognoseintervallen",
       subtitle = "Radio = mittlerer Wert",
       x = "TV-Ausgaben (in 1000 USD)", y = "Sales (in 1000 USD)") +
  theme_minimal() +
  theme(text = element_text(size = 16),
        axis.title = element_text(size = 18),
        axis.text = element_text(size = 14),
```

```
plot.title = element_text(size = 20))
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Regressionslinie mit Konfidenz- und Prognoseintervallen

Radio = mittlerer Wert



Statistische Erklärung:

- Die **blaue Linie** ist die Regressionslinie, die den vorhergesagten Wert von **sales** für verschiedene Werte von TV zeigt, wenn **radio** auf seinem Mittelwert gehalten wird.
- Der **blaue Bereich** ist das 95%-Konfidenzintervall für den Erwartungswert.
- Der **rote Bereich** ist das 95%-Prognoseintervall für einzelne Beobachtungen.
- Das Konfidenzintervall ist am **engsten** in der Nähe des Mittelwerts der Prädiktoren und wird breiter, je weiter man sich davon entfernt.
- Das Prognoseintervall ist **deutlich breiter** als das Konfidenzintervall, da es zusätzlich die Variation der einzelnen Beobachtungen um den Erwartungswert berücksichtigt.
- Die meisten Datenpunkte liegen innerhalb des Prognoseintervalls, was darauf hindeutet, dass das Modell die Daten gut beschreibt.

R-Erklärung:

- Die Funktion `seq()` erzeugt eine Sequenz von Werten.
- Die Funktion `ggplot()` initialisiert ein ggplot-Objekt.
- Die Funktion `geom_point()` fügt Punkte zum Plot hinzu.
- Die Funktion `geom_line()` fügt eine Linie zum Plot hinzu.
- Die Funktion `geom_ribbon()` fügt einen schattierten Bereich zum Plot hinzu, der durch die unteren und oberen Grenzen definiert ist.
- Die Funktion `theme_minimal()` setzt ein minimalistisches Thema für den Plot.

- Die Funktion `theme()` passt verschiedene Aspekte des Plots an, wie Schriftgrößen und -stile.

5 Zusammenfassung

In dieser Vorlesung haben wir die statistische Inferenz in Regressionsmodellen kennengelernt. Die wichtigsten Konzepte waren:

1. **Konfidenzintervalle für Regressionskoeffizienten:** Konfidenzintervalle quantifizieren die Unsicherheit der geschätzten Koeffizienten und geben einen Bereich an, in dem der wahre Parameterwert mit einer bestimmten Wahrscheinlichkeit liegt.
2. **Bootstrap-Methoden:** Die Bootstrap-Methode ist ein Resampling-Verfahren, das es ermöglicht, die Stichprobenverteilung einer Statistik zu schätzen, ohne Annahmen über die zugrundeliegende Verteilung zu treffen.
3. **Signifikanztests:** t-Tests für einzelne Koeffizienten und F-Tests für das Gesamtmodell oder für den Vergleich geschachtelter Modelle ermöglichen es, die statistische Signifikanz der geschätzten Effekte zu beurteilen.
4. **Prognose mit Konfidenz- und Prognoseintervallen:** Konfidenzintervalle für den Erwartungswert und Prognoseintervalle für einzelne Beobachtungen quantifizieren die Unsicherheit der Vorhersagen.

Die statistische Inferenz ist ein wichtiger Bestandteil der Regressionsanalyse, da sie es ermöglicht, von der Stichprobe auf die Grundgesamtheit zu schliessen und die Zuverlässigkeit der Schätzungen und Vorhersagen zu beurteilen. Die Bootstrap-Methode bietet eine flexible Alternative zu klassischen parametrischen Methoden, insbesondere wenn die Annahmen der klassischen Methoden nicht erfüllt sind.